Dry Part (50%)

Language Models (20%)

1. First, we calculate the probabilities of the terms in the entire collection to establish the background language model $P(w|C)$.
   - **D1:** {a a a d}
   - **D2:** {a a b c}
   - **D3:** {a d d d}
   - **D4:** {b d d d}

Total terms in collection $(|C|) = 4 + 4 + 4 + 4 = 16$.

- Count of 'a' in C : $3 + 2 + 1 + 0 = 6 \rightarrow P(a|C) = 6/16 = 0.375$
- Count of 'd' in C : $1 + 0 + 3 + 3 = 7 \rightarrow P(d|C) = 7/16 = 0.4375$

Defining the Scoring Formula Ranking by minimizing KL Divergence is equivalent to maximizing the probability of the query given the document model (Query Likelihood). With Dirichlet smoothing, the probability of a word w given a document D is: present in both documents contribute):

$$P(w|D) = \frac{c(w, D) + \mu \cdot P(w|C)}{|D| + \mu}$$

Since the query $q = \{a, d\}$, we calculate the score based on the product of probabilities for terms 'a' and 'd'.

$$core(D) \propto P(a|D) \times P(d|D)$$

$$Score(D) = \frac{c(a, D) + 1000 \cdot 0.375}{|4| + 1000} \cdot \frac{c(d, D) + 1000 \cdot 0.4375}{|4| + 1000}$$

Given that the denominator (1004) is constant and can be ignored for ranking. We only need to compare the numerators:

- D1 {a a a d}: $(c(a) = 3, c(d) = 1)$

$$Score(D1) = (3 + 375) \times (1 + 437.5) = 378 \times 438.5 = \mathbf{165,753}$$

- D2 {a a b c}: $(c(a) = 2, c(d) =)$
$$Score(D2) = (2 + 375) \times (0 + 437.5) = 377 \times 437.5 = \mathbf{164,937.5}$$
- D3 {a d d d): $(c(a) = 1, c(d) =)$
$$Score(D3) = (1 + 375) \times (3 + 437.5) = 376 \times 440.5 = \mathbf{165,628}$$
- D4 {b d d d}: $(c(a) = 0, c(d) = 3)$
$$Score \propto (0 + 375) \times (3 + 437.5) = 375 \times 440.5 = \mathbf{165,187.5}$$

Final Ranking:

$$\mathbf{D_1 > D_3 > D_4 > D_2}$$

2. **To Prove:** The JM smoothed model yields a valid probability mass function (PMF).
The formula:
$$P_{JM}(w|D) = \lambda \cdot P_{MLE}(w|D) + (1 - \lambda) \cdot P(w|C)$$

Requirements & Assumptions:

- $0 \leq \lambda \leq 1$.
- The underlying components are valid distributions:
$$\sum_{w} P_{MLE}(w|D) = 1$$
$$\sum_{w} P(w|C) = 1$$
- Probabilities are non-negative $(P \geq 0)$.

For a function to be a valid PMF, it must satisfy two conditions:

- **Non-negativity:** Since P_{MLE} and $P(C)$ are probabilities (always ≥ 0) and λ is between 0 and 1, the weighted sum is always ≥ 0 .
- **Summation to 1:** We sum the probability over the entire vocabulary V :
$$\sum_{w \in V} P_{JM}(w|D) = \sum_{w \in V} [\lambda P_{MLE}(w|D) + (1 - \lambda) P(w|C)]$$

By linearity of summation:
$$= \lambda \sum_{w \in V} P_{MLE}(w|D) + (1 - \lambda) \sum_{w \in V} P(w|C)$$

Since the components sum to 1:

$$= \lambda(1) + (1-\lambda)(1) = \lambda + 1 - \lambda = 1$$

Conclusion: Since the sum is 1 and all values are non-negative, it is a valid PMF.

3. **To Show:** Ranking by Negative Cross Entropy (-CE) is equivalent to ranking by Query Likelihood.

Definitions:

- **Query Likelihood (Log):** $Score_{QL} = \log P(q|d) = \sum_{w \in q} \log P(w|d)$ (assuming unigram independence).
- **Cross Entropy:** $H(M_q, M_d) = -\sum_{w \in V}^{P} (w|q) \log P(w|d)$.
- **Negative CE:** $-H(M_q, M_d) = \sum_{w \in V}^{P} (w|q) \log P(w|d)$.

Proof:

Let the query language model M_q be the empirical distribution of the query (Maximum Likelihood Estimate).

$$P(w|q) = \frac{c(w,q)}{|q|}$$

Substitute this into the -CE formula:

$$Score_{-CE} = \sum_{w \in V} \frac{c(w,q)}{|q|} \log P(w|d)$$

Terms where w is not in the query ($w \notin q$) have $c(w,q) = 0$, so we only sum over terms in q :

$$Score_{-CE} = \frac{1}{|q|} \sum_{w \in q} c(w,q) \log P(w|d)$$

If we treat the query as a sequence of words, $\sum_{w \in q} c(w,q) \log P(w|d)$ is exactly equivalent to summing $\log P(w|d)$ for every word occurrence in q .

$$Score_{-CE} = \frac{1}{|q|} \log P(q|d)$$

Conclusion: Since $\frac{1}{|q|}$ is a constant for a specific query, it does not change the ranking order. Thus, ranking by -CE is equivalent to ranking by Query Likelihood $(\log P(q|d))$.

Assumption: The "Bag of Words" (Unigram) assumption applies (word independence).

4. **Suggestion: Symmetric KL Divergence** (often called simply "Symmetric KL" or related to Jensen-Shannon Divergence).

   The standard definition for a symmetric version is:

   $D_{Sym}(M_q, M_d) = D_{KL}(M_q||M_d) + D_{KL}(M_d||M_q)$

Why it fits:

- **Symmetry:** $D_{Sym}(A, B) = D_{Sym}(B, A)$ by definition.
- **Non-negative:** Since standard KL divergence is always non-negative ($D_{KL} \geq 0$), the sum of two non-negative numbers is also non-negative.

(Note: Jensen-Shannon Divergence (JSD) is another correct answer, which is a smoothed version of symmetric KL).

5. Using linear interpolation, we can define the new expanded query model $M_{q'}$ as a weighted interpolation of the original query model M_q and the relevant term model M_R.

Formula:

$$P(w|M_{q'}) = \alpha P(w|M_q) + (1 - \alpha)P(w|M_R)$$

Where:

- $P(w|M_q)$ is the probability of the term in the original query (e.g., $\frac{1}{|q|}$ for query terms, 0 otherwise).
- $P(w|M_R)$ is the probability of the term in the relevant set (e.g., $\frac{1}{|R|}$ for relevant terms, 0 otherwise).
- α is a weighting parameter between 0 and 1.

Explanation:

To satisfy the requirement that the model "place greater emphasis on the original query terms," we must set α > 0.5 (or sufficiently high).

This ensures that the probability mass assigned to the original query terms (summing to α) is strictly larger than the mass assigned to the expansion terms (summing to 1-α).

Relevance Models (15%):

1. Assumptions & Formula:
   - **RM3 Formula:** The RM3 model interpolates the original query model (P_{MLE}) with the relevance model (P_{RM1}) derived from the top documents.

$$P_{RM3}(w) = (1 - \beta) \cdot P_{MLE}(w|q) + \beta \cdot P_{RM1}(w)$$

   (Note: We assume $\beta = 0.3$ is the weight of the expansion/feedback component, and \$0.7\$ is the weight of the original query, which is standard practice to prevent drift).

   - **Constant** $P(q|d)$**:** This implies a uniform prior, meaning all retrieved documents contribute equally to the relevance model.

$$P_{RM1}(w) \approx \frac{1}{|D|} \sum_{d \in D} \quad P(w|d)$$

We calculate Document Language Models $\left(P_{MLE}(w|d)\right)$, we calculate the probability of each word in each document based on frequency.

- d_1 **(length 5):** {cat, dog, cow, pig, horse}
  - $P(w|d_1) = 0.2$ for each term.
- d_2 **(length 8):** {the, cat, and, the, dog, are, playing, together}
  - $P(\text{the}|d_2) = 2/8 = 0.25$
  - $P(\text{cat}|d_2) = 1/8 = 0.125$
  - $P(\text{dog}|d_2) = 1/8 = 0.125$
  - Others (and, are, playing, together) = 0.125
- d_3 **(length 6):** {cat, cat, cat, cat, cat, cat}
  - $P(\text{cat}|d_3) = 6/6 = 1.0$

We calculate RM1 Model $(P_{RM1}(w))$Average the probabilities across the 3 documents (since $P(q|d)$ is constant).

$$P_{RM1}(w) = \frac{1}{3}[P(w|d_1) + P(w|d_2) + P(w|d_3)]$$

- **cat:** $\frac{1}{3}(0.2 + 0.125 + 1.0) = \frac{1.325}{3} \approx \mathbf{0.4417}$
- **dog:** $\frac{1}{3}(0.2 + 0.125 + 0) = \frac{0.325}{3} \approx \mathbf{0.1083}$
- **the:** $\frac{1}{3}(0 + 0.25 + 0) = \mathbf{0.0833}$
- **cow, pig, horse:** $\frac{1}{3}(0.2 + 0 + 0) = \mathbf{0.0667}$ (each)
- **and, are, playing, together:** $\frac{1}{3}(0 + 0.125 + 0) = \mathbf{0.0417}$ (each)

We calculate Original Query Model ($P_{MLE}(w|q)$**)** Query q : "cat dog" (length 2)

- $P(\text{cat}|q) = 0.5$
- $P(\text{dog}|q) = 0.5$

Final RM3 Calculation Apply the weighting: $0.7 \cdot \text{Original} + 0.3 \cdot \text{RM1}$.

- $P_{RM3}(\text{cat}): 0.7 \cdot (0.5) + 0.3 \cdot (0.4417) = 0.35 + 0.1325 = \mathbf{0.4825}$
- $P_{RM3}(\text{cat}): 0.7 \cdot (0.5) + 0.3 \cdot (0.4417) = 0.35 + 0.1325 = \mathbf{0.4825}$
- $P_{RM3}(\text{the}): 0.7 \cdot (0) + 0.3 \cdot (0.0833) = \mathbf{0.0250}$
- $P_{RM3}(\text{cow}): 0.78(0) + 0.3 \cdot (0.0667) = \mathbf{0.0200}$ (Similar calculations follow for remaining terms)

Final RM3 Distribution (Top terms):

- cat: 0.4825
- dog: 0.3825
- the: 0.0250
- cow/pig/horse: 0.0200
- and/playing/etc: 0.0125

2. Query drift occurs when the feedback model focuses on non-relevant topics found in the retrieval results. Two ways to prevent this are:

- **Interpolation (Clipping):** Use the RM3 method to linearly interpolate the feedback model with the original query model (giving high weight, e.g., $1-\beta$, to the original query) to ensure the focus remains on the user's original intent.
- **Term Selection/Pruning:** Instead of using all terms from the feedback documents, select only the top k terms with the highest relevance scores or filter terms based on collection frequency (removing high-frequency stop words) to reduce noise.

3. Short queries suffer from "vocabulary mismatch" (e.g., "auto" vs. "car" have zero overlap but same meaning). Therefore, standard word-overlap measures (like Jaccard) often fail.

Method 1: Semantic Vector Similarity (Word Embeddings)

- **Explanation:** Map the queries to a vector space using pre-trained embeddings (like Word2Vec, GloVe, or BERT). You can compute the average vector for each query or use sentence transformers (S-BERT) to get a single vector per query, then calculate the **Cosine Similarity** between the two vectors.
- **Reason Chosen:** This method captures **semantic meaning**. Even if the queries share no common words (e.g., "cheap flights" vs. "budget air travel"), their vectors will be close in the geometric space, correctly identifying them as similar.

Method 2: Expansion-based Similarity (Enrichment)

- **Explanation:** Before comparing, expand both queries using an external resource (like a thesaurus/WordNet) or a large corpus (Pseudo-Relevance Feedback). For example, expand "feline" to {feline, cat, animal} and "cat" to {cat, kitten, feline}. Then, measure the overlap (e.g., using KL Divergence or Dice coefficient) between the **expanded** language models.
- **Reason Chosen:** This overcomes **data sparsity**. By enriching the short text with contextually related terms, we artificially create overlap where there originally was none, allowing standard statistical measures to work effectively.

Passage Retrieval (10%)

here are three distinct approaches to estimate the passage language model $P(q|M_g)$. These approaches focus on leveraging the containing document (D) and the collection (C) to smooth the sparse passage data and address the vocabulary mismatch problem.

Prerequisite Notation:

- g : The specific passage (sequence of text).
- D : The document containing passage g .
- C : The entire collection of documents.
- $P_{MLE}(w|X)$: Maximum Likelihood Estimate of term w in source X (count of w in X divided by length of X).
- V : The vocabulary size.

Approach 1: Three-Component Linear Interpolation (Mixture Model)

This approach extends the standard Jelinek-Mercer smoothing by adding a third component: the document model. It assumes the passage is a mixture of three distinct sources: the passage text itself, the local context (document), and the global background (collection).

The Equation:

$$P(w|M_g) = \lambda_1 P_{MLE}(w|g) + \lambda_2 P_{MLE}(w|D) + \lambda_3 P_{MLE}(w|C)$$

Constraints & Parameters:

- $\lambda_1, \lambda_2, \lambda_3$ are free parameters (weights) such that $0 \leq \lambda_i \leq 1$.
- **Summation Requirement:** $\lambda_1 + \lambda_2 + \lambda_3 = 1$ (Ensures a valid PMF).

Why this works:

- **Vocabulary Mismatch:** If a query term w (e.g., "vehicle") does not appear in the short passage g (which uses "car"), but appears elsewhere in the document D , the term $\lambda_2 P_{MLE}(w|D)$ ensures $P(w|M_g) > 0$.
- **Smoothing:** The document acts as a "local" smoothing agent, which is more specific than the generic collection model, preserving the topic of the passage better than smoothing with$C$ alone.

Approach 2: Hierarchical Dirichlet Smoothing

Instead of mixing all models simultaneously, this approach treats the smoothing hierarchically. We assume the passage is generated from the document model, and the

document is generated from the collection model. We use Bayesian smoothing (Dirichlet) at each stage.

The Equation: This is calculated in two stages:

Stage 1: Smooth the Document Model against the Collection

$$P(w|M_D) = \frac{c(w,D) + \mu_D P_{MLE}(w|C)}{|D| + \mu_D}$$

Stage 2: Smooth the Passage Model against the Smoothed Document Model

$$P(w|M_g) = \frac{c(w,g) + \mu_g P(w|M_D)}{|g| + \mu_g}$$

Notations & Parameters:

- $c(w,X)$: The count of word w in source X .
- $|X|$: The length (total word count) of source X .
- μ_D: Free parameter controlling the amount of smoothing for the document (typically related to average doc length).
- μ_g: Free parameter controlling the amount of smoothing for the passage (typically related to average passage length).

Why this works:

- **Contextual Hierarchy:** It mathematically formalizes the intuition that a passage is a subset of a document. The probability mass for missing terms is "backed off" to the document first.
- **Validity:** Since Dirichlet smoothing produces a valid probability distribution at each step, the final $P(w|M_g)$ is a valid language model.

Approach 3: Proximity-Based Document Smoothing (Kernel Smoothing)

Standard smoothing treats all words in the document D as equally relevant to the passage g . However, words physically closer to the passage (e.g., in the preceding or succeeding paragraph) are likely more relevant than words at the other end of the document. This approach weights the "document context" based on distance.

The Equation:

$$P(w|M_g) = \alpha P_{MLE}(w|g) + (1-\alpha)P_{Local}(w|g, D)$$

Where P_{Local} is the distance-weighted probability from the document:

$$P_{Local}(w|g, D) = \frac{1}{Z}\sum_{i \in D, i \notin g} \quad K\big(dist(i, g)\big) \cdot \mathbb{I}(w_i = w)$$

Notations & Parameters:

- α: Interpolation weight ($0 < \alpha < 1$).
- $dist(i, g)$: The positional distance (number of words) between word i in the document and the center of passage g .
- $K(x)$: A kernel function (e.g., Gaussian e^{-x^2/σ^2} or Triangular) that decreases as distance x increases.
- $\mathbb{I}(w_i = w)$: Identity function (1 if the word at position i is w , 0 otherwise).
- Z : Normalization constant to ensure $\sum_{w} P_{Local} \quad (w) = 1$.
- *(Note: Collection smoothing can be added as a third linear component if needed for non-document terms).*

Why this works:

- **Vocabulary Mismatch:** It pulls in synonyms or related terms from the *immediate context* of the passage. If the passage mentions "he" and the previous sentence identifies "The President", this model captures "President" with high probability, unlike a uniform document model which dilutes it.
- **Precision:** It reduces noise by discounting irrelevant parts of long documents.

<u>Diversification (5%)</u>

The standard and most widely accepted solution for this specific problem in Information Retrieval is **Maximal Marginal Relevance (MMR)**.

The core idea of MMR is to re-rank a set of retrieved documents. Instead of picking the document with the absolute highest relevance score at every step, we pick the document that maximizes a combination of **Relevance to the Query** and **Dissimilarity to the Already Selected Documents**. This creates a "marginal" relevance that decreases as the document becomes more similar to what the user has already seen.

Let:

- q : The user's query.
- R : The set of candidate relevant documents (e.g., the top 50 documents returned by a standard search).
- S : The set of documents already selected to be shown to the user (initially empty).
- $R \setminus S$: The set of candidate documents not yet selected.

The score for a candidate document d_i at each step is calculated as:

$$MMR(d_i) = \lambda \cdot Sim_1(d_i, q) - (1 - \lambda) \cdot \max_{d_j \in S} Sim_2(d_i, d_j)$$

Where:

- $Sim_1(d_i, q)$: The **Relevance Score**. This measures how well document d_i matches the query q . (We will use Cosine Similarity or BM25 here).
- $Sim_2(d_i, d_j)$: The **Redundancy Score**. This measures the content overlap between the candidate document d_i and a document d_j that is already in the selected set S .
- $\max_{d_j \in S}$: We look for the *maximum* similarity between the candidate and any document we have already picked. If a document is highly similar to *even one* selected document, it is penalized heavily.
- λ **(Lambda)**: A diversity parameter between $[0,1]$.
    - If $\lambda = 1$: The system acts like a standard search engine (Pure Relevance).
    - If $\lambda = 0$: The system acts to maximize diversity completely (Pure Novelty).
    - Typically, $\lambda \approx 0.5$ provides a good balance.

To make this concrete, we must define the similarity functions (Sim_1 and Sim_2).

We represent the query q and document d as TF-IDF vectors. The relevance is the cosine of the angle between them:

$$Sim_1(d, q) = \frac{\vec{d} \cdot \vec{q}}{\left|\left|\vec{d}\right|\right| \cdot ||\vec{q}||}$$

We use the same metric to measure the overlap between two documents. If the vectors for d_i and d_j are close (high cosine score), they share many significant words and are likely redundant.

$$Sim_2(d_i, d_j) = \frac{\overrightarrow{d_i} \cdot \overrightarrow{d_j}}{\left|\left|\overrightarrow{d_i}\right|\right| \cdot \left|\left|\overrightarrow{d_j}\right|\right|}$$

The process works iteratively (Greedy Algorithm):

1. **Initialization:**
    a. Start with the set of retrieved documents R .
    b. Create an empty result list $S = \emptyset$.
2. **First Selection:**
    a. For the first document, the "Redundancy" term is 0 (since S is empty).
    b. Select the document with the highest $Sim_1(d, q)$ and add it to S .
3. **Subsequent Selections:**
    a. For each remaining document in R , calculate the MMR score.
    b. The score will be high if the document is relevant to q , but **minus** a penalty if it is similar to anything in S .
    c. Pick the document with the highest MMR score, remove it from R , and add it to S .
4. **Termination:**
    a. Repeat step 3 until you have selected k documents (e.g., the top 10 results).

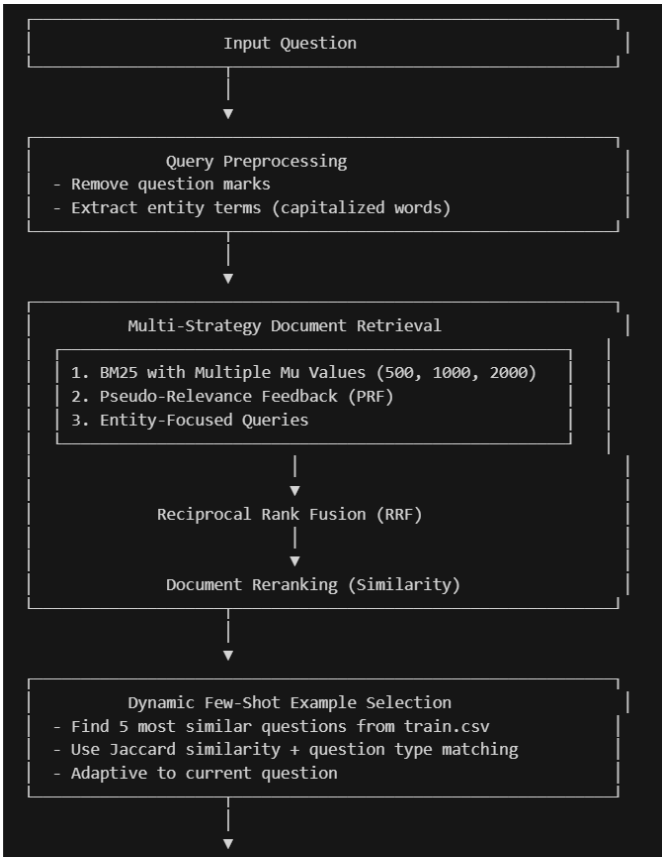
Why This Solves the Problem?

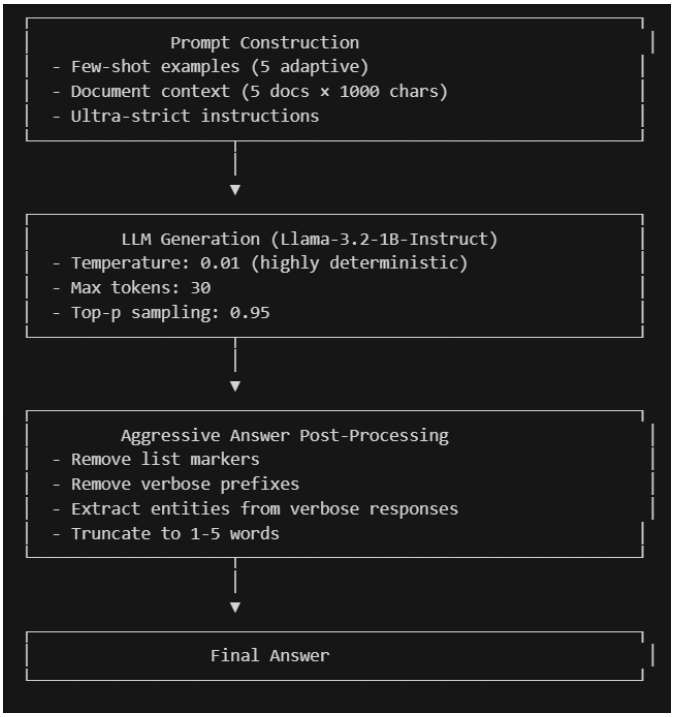
This method directly addresses the prompt's requirements:

- It explicitly includes the **document's relevance score** (λSim_1).
- It explicitly accounts for the **level of redundant content** compared to retrieved documents ($(1 - \lambda)Sim_2$).
- It ensures that if a highly relevant document is effectively a "duplicate" of one already shown, its score drops drastically, allowing a slightly less relevant—but different—document to rise in rank.

Wet part – Kaggle Challenge (50%)

Team Name: Grogu ; Kaggle score: 32.80342


Input Question
Query Preprocessing
- Remove question marks
- Extract entity terms (capitalized words)
Multi-Strategy Document Retrieval
1. BM25 with Multiple Mu Values (500, 1000, 2000)
2. Pseudo-Relevance Feedback (PRF)
3. Entity-Focused Queries
Reciprocal Rank Fusion (RRF)
Document Reranking (Similarity)
Dynamic Few-Shot Example Selection
- Find 5 most similar questions from train.csv
- Use Jaccard similarity + question type matching
- Adaptive to current question


Prompt Construction
- Few-shot examples (5 adaptive)
- Document context (5 docs x 1000 chars)
- Ultra-strict instructions
LLM Generation (Llama-3.2-1B-Instruct)
- Temperature: 0.01 (highly deterministic)
- Max tokens: 30
- Top-p sampling: 0.95
Aggressive Answer Post-Processing
- Remove list markers
- Remove verbose prefixes
- Extract entities from verbose responses
- Truncate to 1-5 words
Final Answer